CHAPTER 12

Flip-Flop Conversion

Make one type of flip-flop from another · the 4-step method · fast tricks · drawn circuits

QUICK TRICK
Learn the two fast tricks first (details below):
1. To make a D flip-flop: D = the *next state* you want. **2. To make a T flip-flop:** T = Q ⊕ (the next state you want).
Most conversions in the exam can be done with these two ideas and a few simple gates.

12.1 What is flip-flop conversion?

● **LEVEL 1 · BEGINNER**

The problem. You have one kind of flip-flop (say a JK) but you need another kind (say a T). **The idea.** Put a **small logic circuit in front** of the flip-flop you have, so that it behaves like the one you need.

Analogy. A travel plug adapter. The wall socket (your flip-flop) stays the same. The adapter (the small logic) changes how the plug (the new inputs) fits.

```
New inputs --->[ Small logic circuit ]---> Inputs of the flip-flop you HAVE ---> Q
                        ^                                                       |
                        +------------------ Q (fed back) --------------------+
```

The new inputs act like the flip-flop you **want**. The small logic is what we have to find.

12.2 The method (4 steps)

● **LEVEL 2 · EXAM**

```
STEP 1  Write the TRUTH TABLE of the flip-flop you WANT (the new one):
        columns = present state Q, new inputs, next state Q+.
STEP 2  For each row, look at the change Q -> Q+.
        Use the EXCITATION TABLE of the flip-flop you HAVE to write
        what its inputs must be.
STEP 3  Simplify each of those inputs with a K-map (use the X's!).
        The variables are the new inputs and Q.
STEP 4  Draw: new inputs -> logic -> old flip-flop.
```

The excitation tables (from Chapter 11). X means “don't care”. Keep this table in front of you:

Q	Q+	SR: S R	JK: J K	D	T
0	0	0 X	0 X	0	0
0	1	1 0	1 X	1	1
1	0	0 1	X 1	0	1
1	1	X 0	X 0	1	0

12.3 Worked example: SR → JK

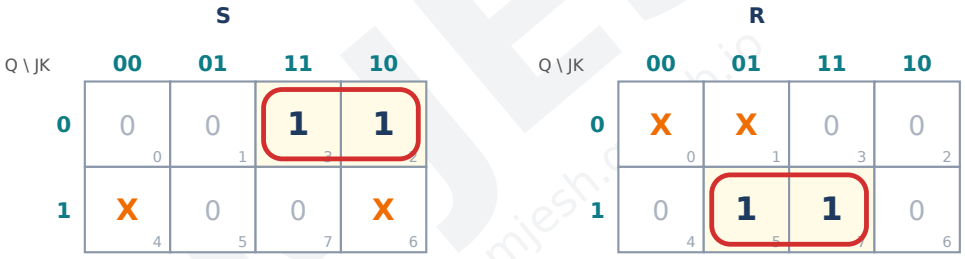
● LEVEL 2 · EXAM

Goal: use an SR flip-flop and make it work like a JK flip-flop. **Why is this nice?** A JK has no invalid case, and an SR does. Our small logic must never give S = R = 1.

Step 1 and 2: the JK truth table (Q, J, K → Q+) and, in the last two columns, the S and R needed for that change (from the excitation table).

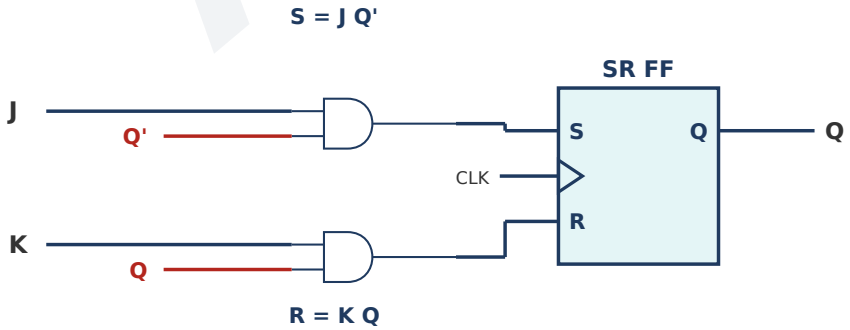
Q	J	K	Q+	S	R
0	0	0	0	0	X
0	0	1	0	0	X
0	1	0	1	1	0
0	1	1	1	1	0
1	0	0	1	X	0
1	0	1	0	0	1
1	1	0	1	X	0
1	1	1	0	0	1

Step 3: K-maps (rows = Q, columns = J K). X cells can be used as 1 to make a bigger group.



S: the group m2, m3 (Q = 0, J = 1, K any) gives **S = J Q'**. **R:** the group m5, m7 (Q = 1, K = 1, J any) gives **R = K Q**.

Step 4: two AND gates in front of the SR flip-flop.



QUICK TRICK

Why is S = R = 1 impossible here? S = JQ' needs Q = 0 and R = KQ needs Q = 1. Q cannot be 0 and 1 at the same time. So the invalid case never happens. ✔

QUICK TRICK

Check with J = K = 1 (toggle): if Q = 0 then S = 1, R = 0 → set → Q = 1. If Q = 1 then S = 0, R = 1 → reset → Q = 0. It toggles. ✔

12.4 Fast tricks (skip the K-map)

● LEVEL 3 · GATE

Think about the **change you want**: the new flip-flop wants to go to a next state Q^+ (a formula in the new inputs and Q). Then:

Flip-flop you HAVE	Its input(s) must be	In words
D	**$D = Q^+$**	Just give it the next state you want.
T	**$T = Q \oplus Q^+$**	T = 1 only when the state has to change.
JK	**$J = Q' \cdot Q^+$ $K = Q \cdot (Q^+)'$**	J: "go up" (from 0 to 1). K: "go down" (from 1 to 0).
SR	**$S = Q' \cdot Q^+$ $R = Q \cdot (Q^+)'$**	Same as JK.

QUICK TRICK

Find Q^+ of the new flip-flop from its characteristic equation (Chapter 11):
SR: $Q^+ = S + R'Q$ | JK: $Q^+ = JQ' + K'Q$ | D: $Q^+ = D$ | T: $Q^+ = T \oplus Q$.
Then put it into the table above. That is the whole conversion!

Example: D → JK (you have D, you want JK)

New flip-flop is JK: $Q^+ = JQ' + K'Q$.

$D = Q^+ \rightarrow$ **D = JQ' + K'Q**. (Two AND gates, one OR gate, one NOT for K'; Q' comes from the flip-flop.)

Example: T → JK (you have T, you want JK)

$T = Q \oplus Q^+ = Q \oplus (JQ' + K'Q)$.

If Q = 0: $Q^+ = J$, so T = J. If Q = 1: $Q^+ = K'$, so T = K. Therefore **T = JQ' + KQ**.

Example: JK → D (the go-up / go-down trick)

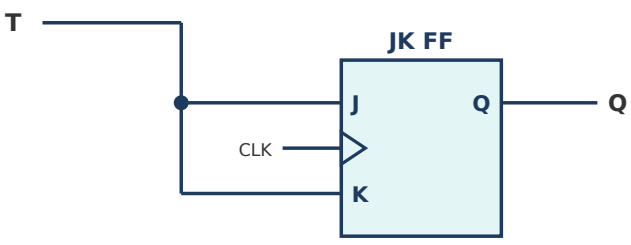
New flip-flop is D: $Q^+ = D$.

J = Q' · D. K = Q · D'. We can simplify: J can be D (when Q = 1 the J input does not matter, it is X) and K can be D' (when Q = 0 K does not matter). **J = D, K = D'**.

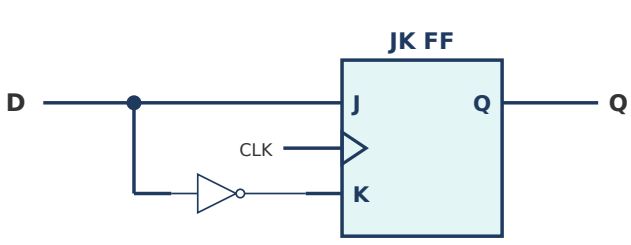
12.5 The four most common conversions (drawn)

● LEVEL 2 · EXAM

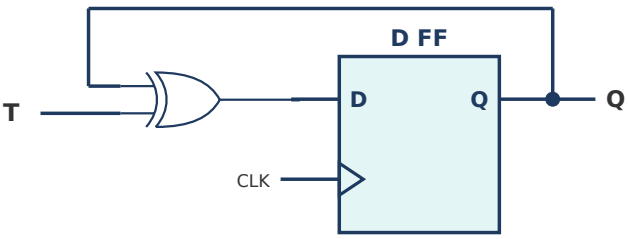
Here are the four conversions you are most likely to be asked about in the exam.



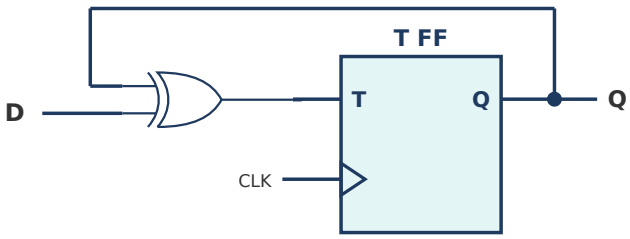
JK → T : J = K = T



JK → D : J = D, K = D'



D → T : D = T ⊕ Q



T → D : T = D ⊕ Q

QUICK TRICK

JK → T needs no extra gate: just join J and K together and call the joint input T. (T = 0: J = K = 0 holds. T = 1: J = K = 1 toggles.)
JK → D needs one NOT gate (J = D, K = D').
D → T and T → D each need one XOR gate (and Q is fed back).

12.6 All conversions in one table

● **LEVEL 2 · EXAM**

Have ↓ Want →	SR	JK	D	T
SR	—	S = JQ', R = KQ	S = D, R = D'	S = TQ', R = TQ
JK	J = S, K = R	—	J = D, K = D'	J = K = T
D	D = S + R'Q	D = JQ' + K'Q	—	D = T ⊕ Q
T	T = SQ' + RQ	T = JQ' + KQ	T = D ⊕ Q	—

Extra gates needed:

Conversion	Extra gates
JK → T	None (join J and K)
JK → D	1 NOT
JK → SR	None (J = S, K = R)
D → T, T → D	1 XOR
SR → JK, SR → T	2 AND
SR → D	1 NOT
D → JK	2 AND + 1 OR + 1 NOT (for K')

GATE POINT

All twelve results in the table were checked by simulation: for every present state Q and every valid input, the converted circuit gives the same next state as the flip-flop it should imitate. (SR = 11 is excluded because it is invalid.)

12.7 Common mistakes

COMMON MISTAKE

- Using the characteristic table of the flip-flop you **have** instead of the one you **want**.
- Forgetting that the old flip-flop's inputs depend on **Q** as well (Q is fed back).
- Using the excitation table of the **new** flip-flop. You need the excitation table of the **old** one.
- Not using the X's (don't cares) in the K-map.
- Mixing up which flip-flop is “given” and which is “wanted”.

QUICK TRICK

Memory line: “Truth table of the **WANTED**, excitation table of the **HAVE**.”

Quick-Solve Tricks — Summary

Situation	Trick
Method	Truth table of the WANTED flip-flop + excitation table of the HAVE flip-flop + K-map.
Have D	D = Q+ (the next state formula of the wanted flip-flop).
Have T	T = Q ⊕ Q+.
Have JK	J = Q'·Q+ (go up), K = Q·(Q+)' (go down). Often J = K = T or J = D, K = D'.
Have SR	S = Q'·Q+, R = Q·(Q+)'.
No extra gate	JK → T (join J, K) and JK → SR (J = S, K = R).
D ↔ T	Always one XOR with Q fed back.
SR → JK	S = JQ', R = KQ. The invalid state never happens.

GATE QUICK CHECK — Chapter 12

```
JK -> T  : J = K = T            JK -> D : J = D, K = D'
D  -> T  : D = T xor Q          T  -> D : T = D xor Q
SR -> JK : S = JQ', R = KQ      D  -> JK: D = JQ' + K'Q
T  -> JK : T = JQ' + KQ         SR -> T : S = TQ', R = TQ
SR -> D  : S = D, R = D'
```

Traps: excitation table of the wrong flip-flop · not feeding Q back · forgetting that JK → T needs **no** gate · D ↔ T needs an XOR (not an AND).

Practice questions

● LEVEL 1 · BEGINNER

Q1. How do you make a T flip-flop from a JK flip-flop?

Q2. How do you make a D flip-flop from a JK flip-flop?

Q3. How many extra gates are needed to make a T flip-flop from a D flip-flop?

● LEVEL 2 · EXAM

Q4. You have a T flip-flop and want a D flip-flop. Write T in terms of D and Q.

Q5. Write S and R for converting an SR flip-flop into a JK flip-flop.

Q6. Which gates are needed to make a JK flip-flop from a D flip-flop?

● **LEVEL 3 · GATE**

Q7. A T flip-flop is made from a D flip-flop with D = T ⊕ Q. If T is held at 0, what does Q do?

Q8. An SR flip-flop is converted to a T flip-flop (S = TQ', R = TQ). If Q = 1 and T = 1, find S, R and the next Q.

Q9. A T flip-flop is converted to a D flip-flop (T = D ⊕ Q). If Q = 1 and D = 0, find T and the next Q.

Q10. A JK flip-flop is built from a D flip-flop (D = JQ' + K'Q). If Q = 1, J = 1, K = 1, find D and the next Q.

Q11. In the SR → JK conversion, why can S = R = 1 never happen?

Q12. Use the go-up/go-down trick: if you want the next state $Q^+ = T \oplus Q$ using a JK flip-flop, find J and K.

Answers

Q	Solution
1	**Join J and K** together and use the joint input as T (J = K = T).
2	**J = D and K = D'** (one NOT gate).
3	**One XOR gate** (D = T ⊕ Q).
4	T = Q ⊕ Q+ and Q+ = D, so **T = D ⊕ Q**.
5	**S = JQ', R = KQ** (two AND gates).
6	D = JQ' + K'Q → **two AND gates, one OR gate, and one NOT** (for K'). Q' is taken from the flip-flop.
7	D = 0 ⊕ Q = Q, so the flip-flop reloads its own value: **Q holds** (no change).
8	S = 1·0 = 0, R = 1·1 = 1 → reset → next Q = **0** (it toggled, as T = 1 should).
9	T = 0 ⊕ 1 = **1** → toggle → next Q = **0** (equals D = 0 ✔).
10	D = 1·0 + 0·1 = **0** → next Q = **0** (J = K = 1 toggles 1 → 0 ✔).
11	S = JQ' is 1 only when Q = 0. R = KQ is 1 only when Q = 1. Both can never be 1 together, because Q cannot be 0 and 1 at once.
12	J = Q'·Q+ = Q'(T ⊕ Q). When Q = 0, T ⊕ Q = T, so J = T. K = Q·(Q+)' = Q·(T ⊕ Q)'. When Q = 1, T ⊕ Q = T', so (Q+)' = T and K = T. Result: **J = K = T** (this is the JK → T conversion).

Next: Chapter 13 — State Machines (state diagram, state table, Moore and Mealy machines, state reduction, sequence detector).